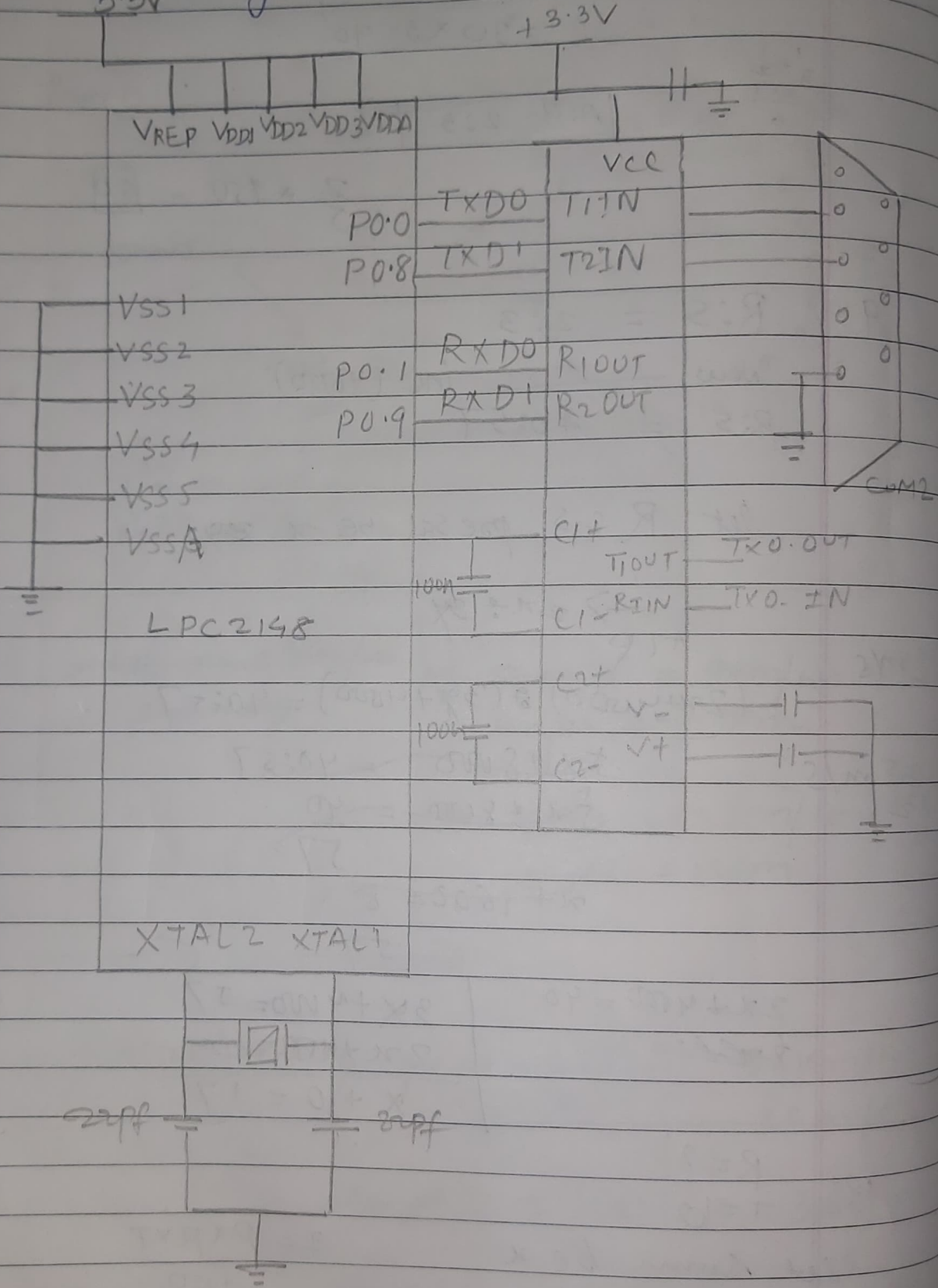


Assignment 04

1. Interfacing diagram of GPS using UART with LPC



Program —

```
#define CR 0x0D  
#include <LPC21XX.H>
```

```
void init_serial(void);  
int put_char(int ch);  
int get_char(void);  
unsigned char test;  
int main(void)
```

```
{  
char *ptr = "*** UART 0 Demo ***\n\n\r type";
```

```
VPBDIV = 0x02
```

```
init_serial();
```

```
while(1)
```

```
{  
while(*ptr)
```

```
{  
putchar(*ptr++);
```

```
}  
putchar(getchar());
```

```
}
```

```
void init_serial()
```

```
{
```

```
PINSEL0 = 0x00000005;
```

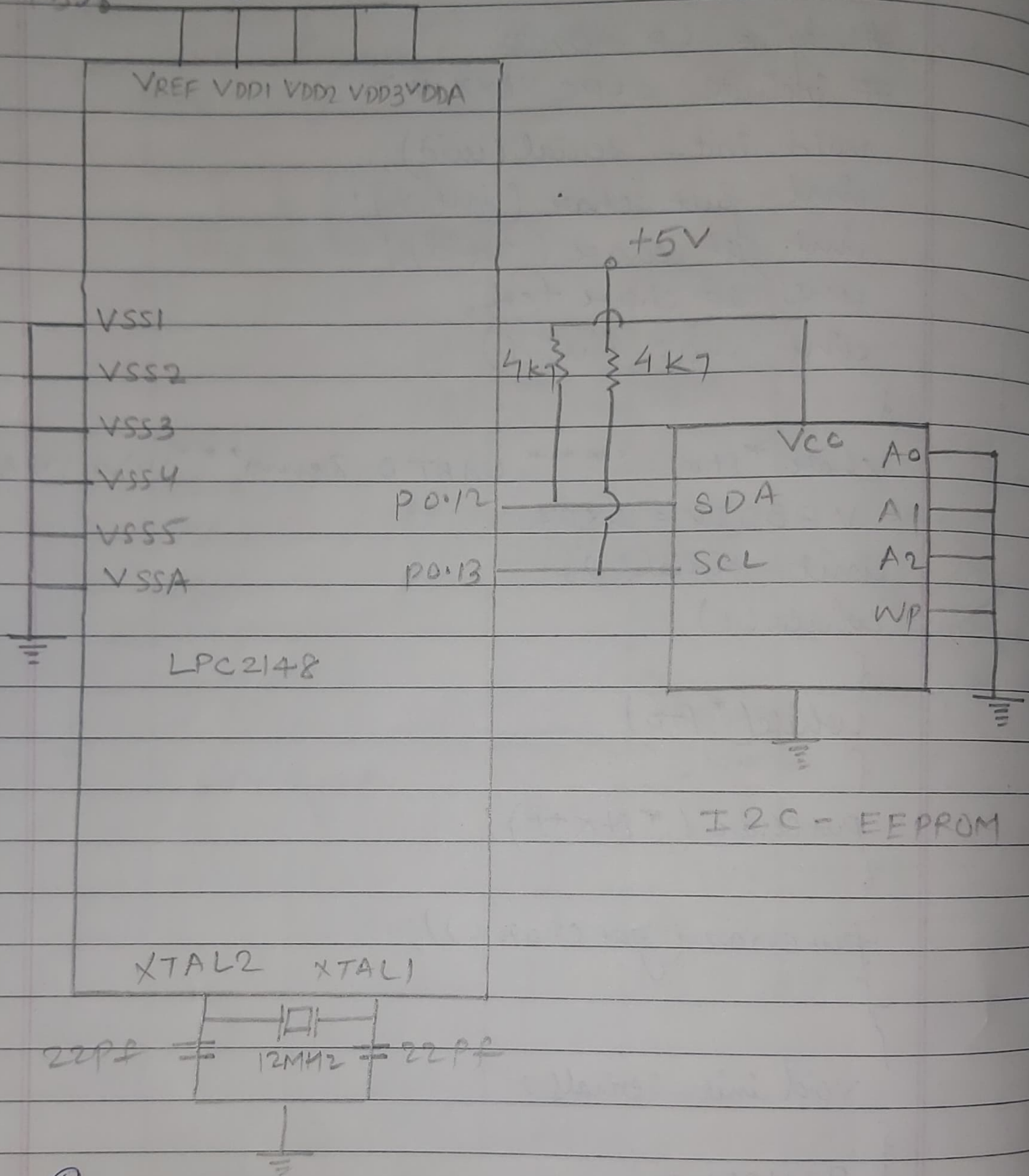
```
UOCLR = 0x00000083;
```

```
UODLL = 0x000000C3;
```

```
}
```

2. Interfacing of I2C EEPROM using LPC2148.

+3.3V



Program:

```
#include <LPC2148.h>
#include "UART.h"
```

```
void I2C_ISR (void) - irq;
void wait (unsigned int);
void I2C_Init (void);
```

Teacher's Signature.....

```

uint I2C_Start ( unsigned int slave - ADDR);
uint I2C_Write ( unsigned char * Buf;
                unsigned int count);

```

```

unsigned char Rec [Max] = 4 "NO-DATA!";
unsigned char index = 0;
unsigned char flag = 0, Ready = 0, Erase = 0;

```

3. C program for on chip ADC for LPC2448

```

→ #include <LPC2448.h>
#include <stdio.h>
#include "LCD.h"
char adcReading[10];
void ADC_Init (void)
{
    PINSEL1 = 0x01000000;
}
unsigned int ADC_GetAdcReading ()
{
    unsigned int adcData;
    ADC_R = 0x01200302;
    while (1((adcData = ADGDP) & 0x80000000))
    }
    return ((adcData >> 6) & 0x3ff);
}
int main ()
{
    unsigned int delay, adc;
    ADC_Init ();
    led_Init ();
    UART0_Init ();
    DisplayRow (1, "ADC Test");
    while(1)

```

```

{
    adc = ADC - Get Adc reading();
    printf (adc(reading, " ACD 0Ch = 0x%.03x", adc);
    Display Row (2, adc reading);
    for (delay = 0; delay < 6000; delay += 1);
    printf ("adc data %d", adc);
}
}

```

4. (1) I2C :-

a. START → is set to enter master mode & start condition

b. STOP → Sends stop condition to master mode & recovers from an error in slave mode

(2) VIC :-

(i) It takes 32 interrupt request i/p & programmable assigns them into three categories, FIQ, vectored IRQ & non vectored IRQ.

(ii) FIQ - high priority, vectored IRQ middle priority, non vectored IRQ - low priority

(3) (i) SPI protocol :-

(i) SPI is b/w / firmware communication protocol developed by Motorola

(ii) SPI bus is simple 4 wire serial communication interface used by many microprocessors / micro peripheral

chips that enables controller & peripheral devices

(4) Featuring of ADC :-

- 10 bit Analog to digital converter
- Input multiplexing among 6008 pins.
- power-down mode
- measurement range 0 to V_{ref}
- 10bit conversion time $\times 2.44 \mu s$
- burst conversion mode

5. Interfacing diagram of SD card :-

```
#include <LPC24xx.h>
```

```
#include <type.h>
```

```
#include "spi_mmc.h"
```

```
BYTE MMCWRData[MMC DATA-SIZE];
```

```
BYTE MMCRData[MMC DATA-SIZE];
```

```
BYTE MMCCmd[MMC CMD-SIZE];
```

```
BYTE MMCStatus = 0;
```

```
void SPI_Init ( )
```

```
{
```

```
    DWORD - port config ;
```

```
    BYTE i, Runny;
```

```
    Port fconfig = PINSEL1;
```

```
    PINSEL1 = Port config 10x00AS;
```

```
    IODIRO = SPI_SEL;
```

```
    IOSET0 = SPI_SEL;
```

6. Compare

I2C	SPI
(i) I2C originates from Philips division	(i) SPI originates from Motorola
(ii) I2C allows multiple master & slaves on the bus	(ii) SPI work only with one master device
(iii) It is half-duplex	(iii) Full duplex
(iv) It has only two lines SDA, SCL	(iv) SCLK, SS, MISO, MOSI
(v) 10 Kbps to 1 Kbps delay	(v) 10 Mbps to 20 Mbps
(vi) more complex	(vi) less complex

Assignment 5

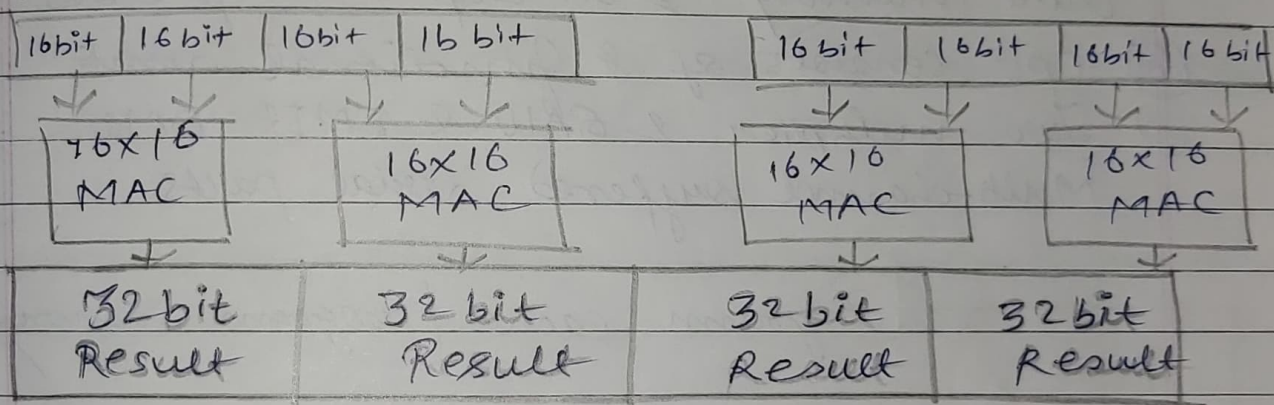
1. Explain:

1.7 SIMD :-

Single instruction multiple data processor

(i) This is used to increase the no. of operations performed per instruction.

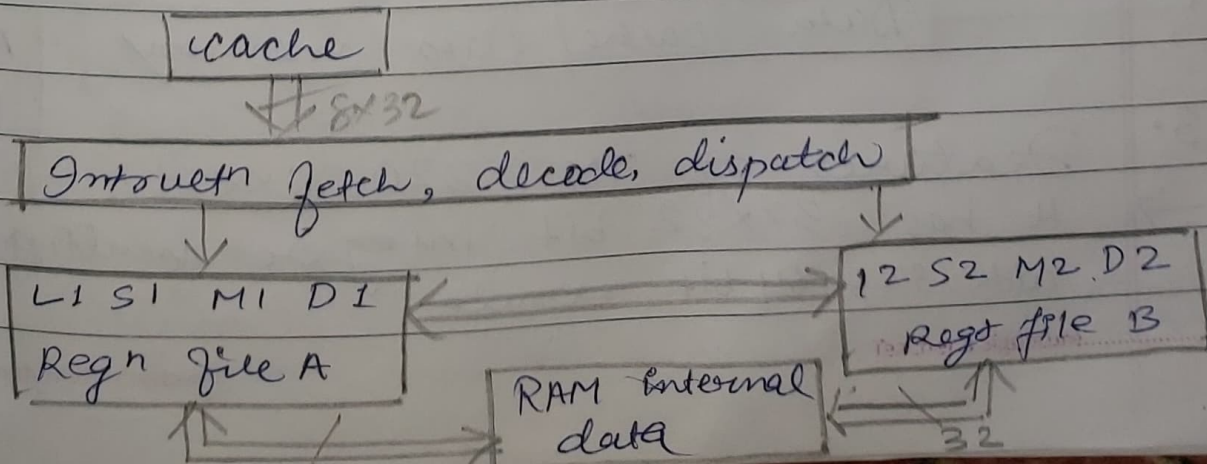
(ii) The processor has multiple data paths & multiple execution units in DSP processor with SIMD architecture



2. VLIW :-

Very Long Instruction Word.

(i) It is to increase no. of instructions that are processed per cycle & concentration of several short inst & reg multiple execution units running in parallel carry out the instructions single cycle

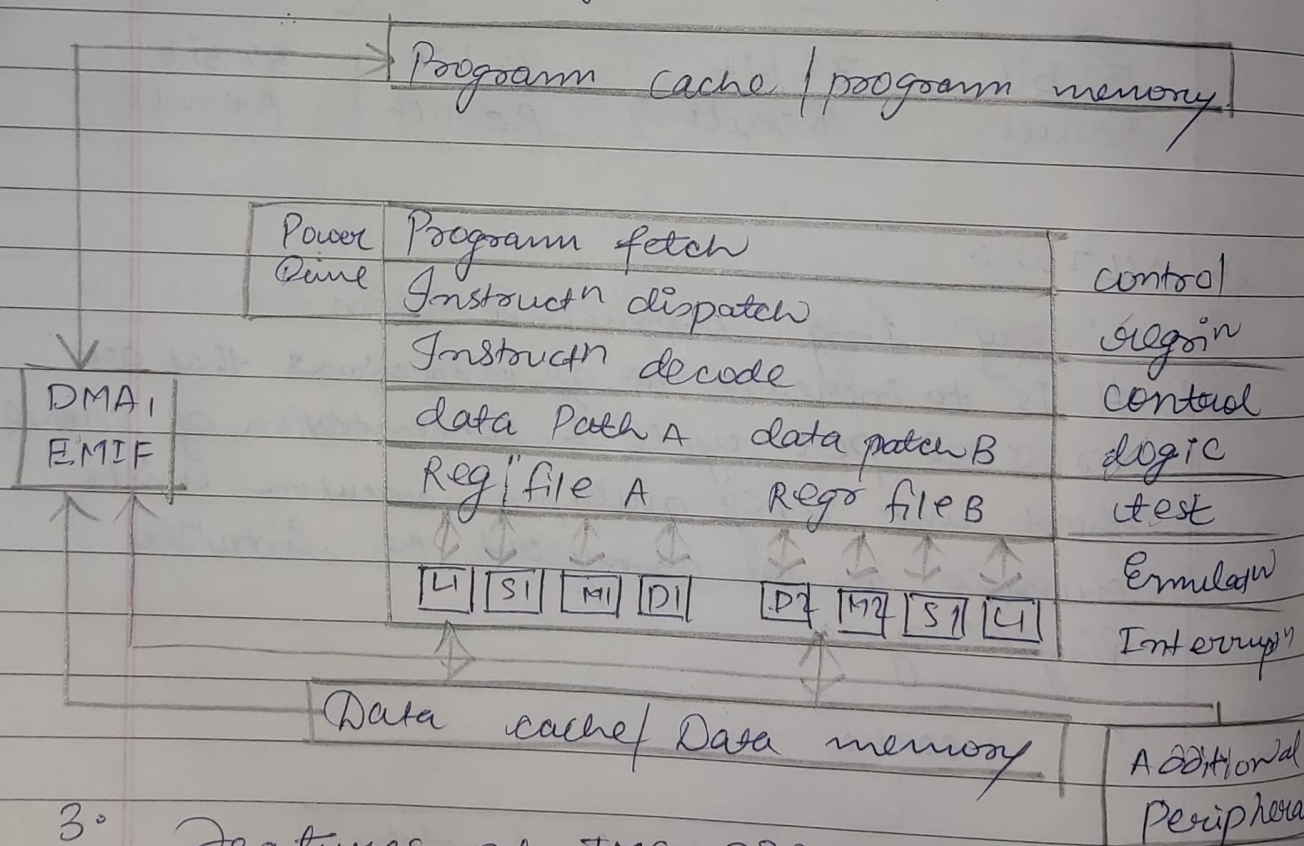


3> Super scalar processors:-

- (i) Superscalar processor is a CPU that implements a form of parallelism called instruction level parallelism within a single processor.
- (ii) A single core superscalar processor is classified as a SIMD processor.

2. Architecture of TMS 320C67X DSP processor.

- (i) The TMS320C67X DSP use the VLIW architecture.
- (ii) It contains of 32 bit CPU on chip program data memory & on-chip peripherals.
- (iii) CPU consists of 8 functional units.
- (iv) Two multipliers & SALU & EMIF DMS, Multi-channel buffered serial ports



3. Features of TMS 320C6xx:-

- It has 32x32 bit integer multiplication with 32 or 64 bit result

- Supports 8/16/32 bit data formats.
- Advanced VLIW CPU with ALUs
- Instructions packaging reduces code size, program fetches & powers assumption.
- Single precision & double precision.

4. General purpose reg-

- (i) There are 2 GPI as file A & B
- (ii) Each reg^r file contains 16 no. of 32 bit reg^r
A0-A15 reg^r for reg A & B0-B15 reg^r for Reg B.
- (iii) There are 32 no. of 32 bit reg^r for each path A0-A31 → Regs A B0-B31 for Reg B

Device family	Register pairs
① C62X / C64X C67X	A1:A0 B1:B0 A15:A14 B15:B14
② C64 only	A17:A16 B17:B16 A31:A30 B31:B30

5. Data paths in TMS320C67XX.

→ There are two data paths →

(1) Data path A :- (2) Data path B

(i) There are 3 different path in TMS 320C67X CPU device

- 1) Reg^r file data path
- 2) Reg^r file cross path
- 3) Reg^r file memory access path.

Data lines are 32 bit but sometimes supports 40 bit & 64 bit double word.

6. Explain Selection criteria of DSP

→ (i) Architectural features —

Size of on-chip memory.

Special instruction. I/O capability.

(ii) Execution Speed —

clock speed in MHz & no. of instructions in MIPS & MFLOPS

(iii) Type of arithmetic —

i) fixed point

ii) floating point

word length — It determines how accurately parameter & result of DSP operations can be presented.

7. Control Regs file:

(i) It contains 10 control regs.

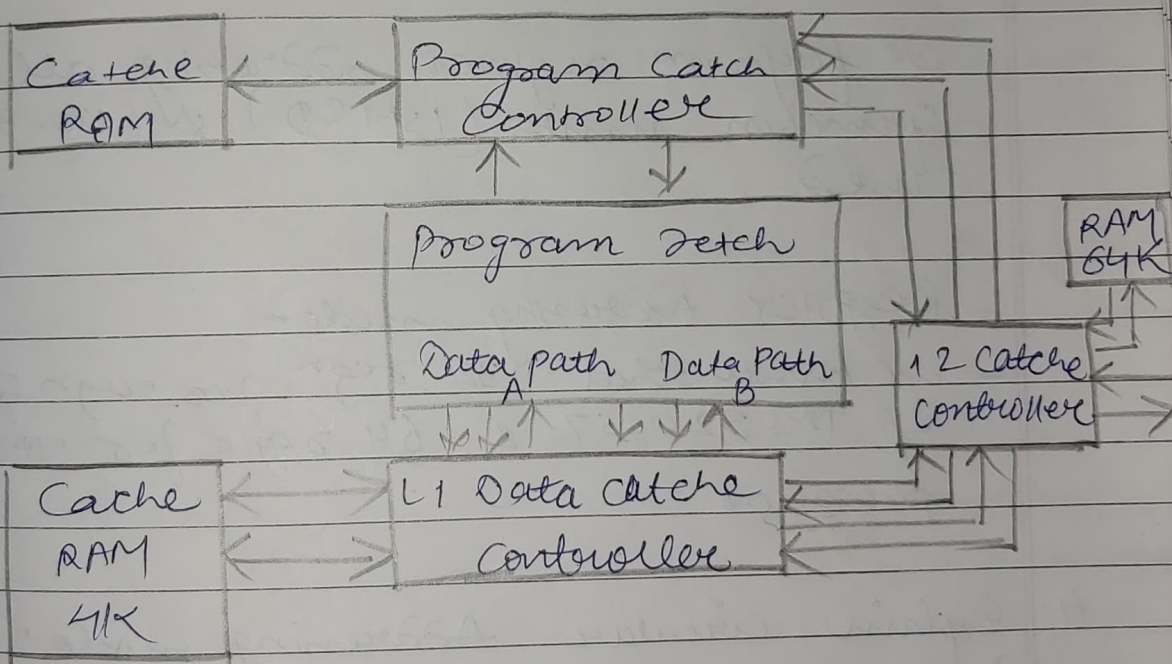
(ii) SZ unit can read & write to control regs file.

(iii) They are accessed by the more b/w the control file & Regs (MVC) instruction but some of the control Regs unit are specially accessed in other way.

Assignment 6

1. Internal memory details of C6x processor

- (i) The size of internal data memory is 64K bytes of RAM.
- (ii) DMA bytes of RAM controlled & CPU can access internal data memory space through data memory controller.
- (iii) 256 wide path is provided from to the CPU to allow a continuous stream & light 32 bit encoder from maximum performance



2. On-chip peripheral in TMS 320C67xx?

- The programmable on-chip peripherals of TMS 320C67x processor are as →
- Two multichannel buffered serial ports
- Two 32 bit timers
- DMA/EDMA

- EMIF
- HPI
- Boot config
- Interrupt sector
- Expansion bus
- power down logic

3. Addressing modes supported by C67 processor

- (i) Register addressing mode
- (ii) Indirect addressing mode
- (iii) Circular addressing mode

To perform register addressing all the function units (ISM & D) & (130-315) are used

Register Addressing mode -

- we know that 32 regn for regn file of TMS320C67 & 64 reg's for TMS320C64 processor

4. Explain circular Addressing mode

- (i) D₁ unit of rego path A & D₂ of path B is used in this mode
- (ii) The regn A4-A7 & B4-B7 can also be used
- (iii) Syntax -
mnemonic unit mode field, regn
- (iv) The instruction contains 4 fields as
 - mnemonic
 - functional unit

→ Mode field
→ destination field

5. Explain fixed point instruction -

→ move instruction -

To move the contents b/w regs of the regn file & upper bits of regn me.

Instruction	functional unit	Desc
MV	L1 or L2 D1 or D2	move values within regn

MVC	S2 only	move b/w control regn
-----	---------	-----------------------

27 Load/store instruction:-

The difference type load/store instructions are based on byte, half word & word access

Instruction	functional unit	desc
LD3/STB	D1 or D2	load byte from memory to regn
LDBV	D1 or D2	memory to regn

37 Add instruction:-

Regn addressing mode is used for all add instrn except circular addressing mode is used for additn addressing mode

Instn
ADDL

Functional unit
L1 or L2

desc
unsign
int
with

4) Subtract instructions:
SUBC L1 or L2

condit
int
sub
← Shift
for di

7. C program for linear convolution

```
#include <stdio.h>
```

```
int x[15], h[15], y[15];
```

```
void main()
```

```
{  
    int i, j, m, n;
```

```
    pf("Enter volume for m");  
    sf("%d", &m);
```

```
    pf("Enter value for n");  
    sf("%d", &n);
```

```
    pf("Enter value for i/p x(n)");  
    for(i=0; i<m; i++)
```

```
        sf("%d", &x[i]);
```

```
    pf("Enter value for i/p h(n)");  
    for(i=0; i<n; i++)
```

```
        sf("%d", &h[i]);
```

```
    for(i=m; i<=m+n-1; i++)
```

```
        x[i]=0;
```

```
    for(i=n; i<=m+n-1; i++)
```

```
        y[i]=0;
```

Teacher's Signature

PAGE NO.: _____
DATE: / /

```
for (i=n; i<=m+n-1; i++)
    h[i] = 0;
```

```
{
    for (i=0; i<=m+n-1; i++)
```

```
        y[i] = 0;
```

```
        for (j=0; j<=i; j++)
```

```
            y[i] = y[i] + (x[j] * h[i-j]);
```

```
        }
    }
    for (i=0; i<=m+n-1; i++)
```

```
        pf ("In the value of o/p
            y[%d] = %d, i, y[i]);
```

```
    }
```